



[Draft] NEXT Squad: chat conversation from Microsoft Teams

From mohamed.berro@lmco.com
Draft saved Thu 8/27/2026 12:00 PM

Microsoft Teams



Johnson, Charles P (US) 18 hours ago

I got the navigator llm to spit this out. Feel free to use it.

Below is a **stand-alone Python module** named `guidance_lib.py` that can be fed directly to **pySINDy** as a custom function library.
All functions accept a **state vector**

```
s = [x, y, z, vx, vy, vz] # Cartesian position (m) and velocity (m s-1)
```

and return a **control-acceleration vector** (3 × 1) that you can treat as part of the right-hand side of the dynamics:

$$\begin{bmatrix} \dot{\mathbf{s}} \end{bmatrix} = \begin{bmatrix} \mathbf{v} \\ \mathbf{a} \end{bmatrix} \begin{bmatrix} \text{gravity} \\ \mathbf{a}_{\text{guidance}}(\mathbf{s}, \dots) \end{bmatrix}$$

The module groups the laws discussed earlier:

Category	Function name	Brief description
Basic	gamma_hold	Keeps flight-path angle (γ) constant.
	gamma_pursuit	Drives γ toward a commanded value (γ_cmd).
	pro_nav	Classical proportional navigation (PN) with navigation constant N.
Extended / Robust	augmented_pro_nav	PN + LOS-angle bias term.
	constant_bearing	Maintain constant LOS bearing (zero-bearing-rate).
	lead_angle	Fixed lead angle ahead of LOS.
	zero_effort_miss	ZEM/ZEV-style optimum intercept.
	sliding_mode_guidance	Sliding-mode law with reaching gain k_s.
	energy_based_guidance	Regulates specific mechanical energy toward a set-point.
Rendezvous	cw_lqr	LQR on Clohessy-Wilkinson linear relative dynamics.
	cw_lyapunov	Lyapunov-based rendezvous law.

mpc_rendezvous

Very light-weight "single-step" MPC
(quadratic program) that can be
expanded for full-horizon MPC later.

1. Full source (guidance_lib.py)

```
# guidance_lib.py
# -----
# Function library for pySINDy – guidance / interception laws
# -----
# State vector (input to every function):
#   s = [x, y, z, vx, vy, vz]   # meters, meters/second
#
# All functions return a NumPy array of shape (3,) that
#   represents the commanded acceleration (m/s^2)
#
# -----
import numpy as np
from numpy.linalg import norm

# -----
# Helper utilities (shared by many laws)
# -----
def los_vector(s_target, s_own):
    """Line-of-sight unit vector from ownship to target (target – own)."""
    r = s_target[:3] - s_own[:3]
    return r / norm(r)

def los_rate(s_target, s_own):
    """Relative LOS angular rate (rad/s) using velocity difference."""
    r = s_target[:3] - s_own[:3]
    v_rel = s_target[3:6] - s_own[3:6]
    return np.cross(r, v_rel) / (norm(r) ** 2)

def flight_path_angle(v):
    """γ = atan2(v_z, sqrt(v_x^2+v_y^2)) (negative for descent)."""
    vh = np.linalg.norm(v[:2])
    return np.arctan2(v[2], vh)

# -----
# 1) Basic guidance laws
# -----
def gamma_hold(s, gamma_ref=np.radians(5.0), k_gamma=0.1):
    """
    Hold a constant flight-path angle γ_ref.
    Simple PD on the error between current γ and γ_ref.
    """
    v = s[3:6]
    gamma = flight_path_angle(v)
    error = gamma - gamma_ref
    # Normal-acceleration needed to change γ
    a_mag = -k_gamma * error * norm(v)   # m/s^2
    # Direction: rotate velocity vector toward/away from vertical
    # Compute a unit vector perpendicular to velocity in the vertical
    plane
    vh = np.linalg.norm(v[:2])
    if vh == 0:
        perp = np.array([0, 0, 0])
    else:
        horiz = v[:2] / vh
        perp = np.array([-horiz[0] * np.sin(gamma),
                        -horiz[1] * np.sin(gamma),
                        np.cos(gamma)])
    return a_mag * perp

def gamma_pursuit(s, gamma_cmd, k_gamma=0.2):
```

```

"""
Pursue a commanded flight-path angle  $\gamma_{cmd}$ .
"""
v = s[3:6]
gamma = flight_path_angle(v)
error = gamma_cmd - gamma
a_mag = k_gamma * error * norm(v)
# Same perpendicular direction as in gamma_hold
vh = np.linalg.norm(v[2:])
if vh == 0:
    perp = np.array([0., 0., 0.])
else:
    horiz = v[2:] / vh
    perp = np.array([-horiz[0] * np.sin(gamma),
                    -horiz[1] * np.sin(gamma),
                    np.cos(gamma)])
    return a_mag * perp

def pro_nav(s_own, s_target, N=4.0, k_bias=0.0):
    """
    Classical Proportional Navigation (PN).
    a = N * V_c * lambda_dot + k_bias * LOS
    where V_c is the closing velocity magnitude.
    """
    lambda_vec = los_vector(s_target, s_own)           # unit LOS
    lambda_dot = los_rate(s_target, s_own)             # rad/s vector
    v_rel = s_target[3:6] - s_own[3:6]
    Vc = -np.dot(v_rel, lambda_vec)                   # closing speed
    (>0 if closing)
    a_cmd = N * Vc * np.cross(lambda_dot, lambda_vec)  # classic PN
    term
    a_cmd += k_bias * lambda_vec                      # optional bias
    / lead term
    return a_cmd

# -----
# 2) Extended / robust laws
# -----
def augmented_pro_nav(s_own, s_target, N=4.0, k_bias=0.1):
    """
    PN with a constant LOS-angle bias (helps against target maneuvers).
    """
    return pro_nav(s_own, s_target, N=N, k_bias=k_bias)

def constant_bearing(s_own, s_target, k_cb=0.5):
    """
    Keep LOS bearing constant (zero bearing-rate).
    a = -k_cb * (d/dt LOS bearing) * Vc * lambda_hat
    """
    lambda_vec = los_vector(s_target, s_own)
    lambda_dot = los_rate(s_target, s_own)
    # Bearing rate magnitude (project onto plane orthogonal to LOS)
    bearing_rate = np.linalg.norm(lambda_dot - np.dot(lambda_dot,
    lambda_vec) * lambda_vec)
    v_rel = s_target[3:6] - s_own[3:6]
    Vc = -np.dot(v_rel, lambda_vec)
    return -k_cb * bearing_rate * Vc * lambda_vec

def lead_angle(s_own, s_target, psi_lead=np.radians(15)):
    """
    Fixed lead-angle guidance.
    The commanded LOS is rotated by  $\psi_{lead}$  about the axis normal to the
    LOS-plane.
    """
    lambda_vec = los_vector(s_target, s_own)
    # Build a rotation that spins lambda_vec by psi_lead around a vector
    # orthogonal to both lambda_vec and the vertical (z) axis to keep the
    rotation in the LOS plane.
    # Choose an arbitrary reference (e.g., world x-axis) unless it aligns
    with lambda.

```

```

    ref = np.array([1., 0., 0.])
    if np.allclose(np.cross(lambda_vec, ref), 0):
        ref = np.array([0., 1., 0.])
    axis = np.cross(lambda_vec, ref)
    axis = axis / norm(axis)
    # Rodrigues rotation formula
    cosψ = np.cos(psi_lead)
    sinψ = np.sin(psi_lead)
    lambda_lead = (cosψ * lambda_vec +
                   sinψ * np.cross(axis, lambda_vec) +
                   (1 - cosψ) * np.dot(lambda_vec, axis) * axis)
    # Accelerate toward the rotated LOS direction
    k = 0.3 # simple proportional gain
    return k * lambda_lead

def zero_effort_miss(s_own, s_target, t_go=5.0, k_zem=1.0):
    """
    Zero-Effort-Miss (ZEM) / Zero-Effort-Velocity (ZEV) law.
    Assumes straight-line constant-velocity motion over a predicted
    time-to-go.
    """
    r = s_target[:3] - s_own[:3]
    v_rel = s_target[3:6] - s_own[3:6]
    # Desired acceleration to null the predicted miss at t_go
    a_cmd = ( -2.0 * (r + v_rel * t_go) ) / (t_go ** 2)
    return k_zem * a_cmd

def sliding_mode_guidance(s_own, s_target, k_s=0.8, epsilon=0.01):
    """
    Simple sliding-mode law on the LOS angle error.
    a = -k_s * sat(σ/ε) * lambda_hat
    where σ = LOS angle error (here approximated by LOS rate magnitude).
    """
    lambda_vec = los_vector(s_target, s_own)
    lambda_dot = los_rate(s_target, s_own)
    sigma = np.linalg.norm(lambda_dot) # surrogate for angle error rate
    sat = np.clip(sigma / epsilon, -1.0, 1.0)
    return -k_s * sat * lambda_vec

def energy_based_guidance(s_own, s_target, E_ref=0.0, k_e=0.05):
    """
    Regulate specific mechanical energy (kinetic + potential) to a
    reference.
    For simplicity we ignore gravity potential (assume short arcs) and
    just control kinetic energy.
    """
    v = s_own[3:6]
    E = 0.5 * np.dot(v, v) # specific kinetic energy
    error = E - E_ref
    # Acceleration opposite to velocity reduces energy, parallel increases
    it.
    a_cmd = -k_e * error * v / (norm(v) + 1e-6)
    return a_cmd

# -----
# 3) Rendezvous-focused laws
# -----

def cw_relative_state(s_own, s_target):
    """
    Compute the relative state in the Hill (LVLH) frame assuming a
    circular
    reference orbit for the target.
    This function is a thin wrapper – for a full implementation you need
    the
    target orbital angular rate (n). Here we assume n = 0 (transfer-frame
    only).
    """
    # For many quick-look SINDy experiments we treat the LVLH frame as
    inertial.
    dr = s_target[:3] - s_own[:3]

```

```

dv = s_target[3:6] - s_own[3:6]
return np.concatenate([dr, dv])

def cw_lqr(s_own, s_target, Q=None, R=None):
    """
    LQR on the classic 6-state CW linear system.
    The user can optionally supply Q and R weighting matrices; defaults
    are
    diagonal with moderate values.
    Returns a 3-vector acceleration to be added to the relative dynamics.
    """
    # CW continuous-time A matrix for circular orbit with mean motion n
    # For a generic short-range rendezvous we can approximate  $n \approx 0$ .
    # Using  $n = 0$  reduces A to double integrator (simple PD).
    # If you have the orbital rate, replace n below.
    n = 0.0
    A = np.block([
        [np.zeros((3, 3)), np.eye(3)],
        [3 * n**2 * np.eye(3), np.zeros((3, 3))]
    ])
    B = np.block([
        [np.zeros((3, 3))],
        [np.eye(3)]
    ])

    # Default weighting
    if Q is None:
        Q = np.diag([10., 10., 10., 1., 1., 1.])
    if R is None:
        R = np.diag([0.1, 0.1, 0.1])

    # Solve continuous algebraic Riccati equation (CARE)
    # Since we cannot import scipy in the minimal library, we provide a
    simple
    # analytical PD solution that mimics LQR for the double-integrator
    case:
        Kp = np.sqrt(Q[0,0] / R[0,0]) # position gain
        Kd = np.sqrt(Q[3,3] / R[0,0]) # velocity gain
        # Build state-feedback law:  $u = -Kp * r - Kd * v_{rel}$ 
        rel = cw_relative_state(s_own, s_target)
        r = rel[:3]
        v_rel = rel[3:6]
        a_cmd = -Kp * r - Kd * v_rel
        return a_cmd

def cw_lyapunov(s_own, s_target, k=0.5):
    """
    Simple Lyapunov-based law:  $u = -k * r - k * v_{rel}$ 
    Guarantees  $\dot{V} = -k (||r||^2 + ||v||^2) < 0$ .
    """
    rel = cw_relative_state(s_own, s_target)
    r = rel[:3]
    v_rel = rel[3:6]
    return -k * (r + v_rel)

def mpc_rendezvous(s_own, s_target, horizon=10, dt=1.0, Q=None, R=None):
    """
    A *very* lightweight, single-step "explicit MPC" implementation.
    It solves a quadratic program of the form:
        min    0.5*u.T*R*u + (x_pred).T*Q*x_pred
    subject to thrust magnitude limits.
    For a full multi-step MPC you would embed this inside a loop.
    """
    # Relative state
    rel = cw_relative_state(s_own, s_target)
    r, v_rel = rel[:3], rel[3:6]

    # Prediction (single-step Euler)
    # x_next = A*rel + B*u (A/B from CW with n=0)
    A = np.block([[np.eye(3), dt * np.eye(3)],

```

```

        [np.zeros((3,3)), np.eye(3)])
B = np.block([[0.5 * dt**2 * np.eye(3)],
              [dt * np.eye(3)]])

# Default weights
if Q is None:
    Q = np.diag([1., 1., 1., 0.1, 0.1, 0.1])
if R is None:
    R = np.diag([0.01, 0.01, 0.01])

# Closed-form solution of the unconstrained QP:
# u* = -(B^T Q B + R)^(-1) B^T Q (A*rel)
H = B.T @ Q @ B + R
f = B.T @ Q @ (A @ rel)
u_star = -np.linalg.solve(H, f)

# Apply a simple magnitude saturation (e.g., 0.2 g max)
a_max = 0.2 * 9.80665
if norm(u_star) > a_max:
    u_star = u_star * a_max / norm(u_star)

return u_star

# -----
# Export list for pySINDy CustomLibrary (optional convenience)
# -----
__all__ = [
    # Basic
    "gamma_hold", "gamma_pursuit", "pro_nav",
    # Extended
    "augmented_pro_nav", "constant_bearing", "lead_angle",
    "zero_effort_miss", "sliding_mode_guidance", "energy_based_guidance",
    # Rendezvous
    "cw_lqr", "cw_lyapunov", "mpc_rendezvous"
]

```

2. How to plug the library into pySINDy

```

import numpy as np
import pysindy as ps
from guidance_lib import __all__ as guidance_funcs # import the symbols
import guidance_lib as gl

# -----
# 1) Build a CustomLibrary from the functions above
# -----
library = ps.CustomLibrary(
    library_functions=[gl.gamma_hold,
                      gl.gamma_pursuit,
                      gl.pro_nav,
                      gl.augmented_pro_nav,
                      gl.constant_bearing,
                      gl.lead_angle,
                      gl.zero_effort_miss,
                      gl.sliding_mode_guidance,
                      gl.energy_based_guidance,
                      gl.cw_lqr,
                      gl.cw_lyapunov,
                      gl.mpc_rendezvous],
    function_names=[f.__name__ for f in [
        gl.gamma_hold, gl.gamma_pursuit, gl.pro_nav, gl.augmented_pro_nav,
        gl.constant_bearing, gl.lead_angle, gl.zero_effort_miss,
        gl.sliding_mode_guidance, gl.energy_based_guidance,
        gl.cw_lqr, gl.cw_lyapunov, gl.mpc_rendezvous]],
    # arguments that each function expects (state + optional params)

```

```

    input_features=ps.FiniteDifference(order=2)
)

# -----
# 2) Prepare data (example: simulated 3-D flight)
# -----
# t, X where X has shape (n_samples, 6) = [x,y,z,vx,vy,vz]
t = np.linspace(0, 120, 1201)          # 0-120 s, 0.1 s step
X = np.random.randn(len(t), 6) * 0.0    # replace with real trajectory

# -----
# 3) Fit SINDy
# -----
model = ps.SINDy(
    optimizer=ps.STLSQ(threshold=0.1),
    feature_library=library,
    differentiation_method=ps.FiniteDifference(order=2)
)

model.fit(X, t=t)
model.print()

```

- The CustomLibrary automatically evaluates each guidance function on the **state matrix** X.
- If a function needs extra arguments (e.g., a navigation constant N), you can either **wrap it** with `functools.partial` before passing it to the library, or modify the function to pull the parameter from a global configuration dictionary.

3. Quick usage notes

Situation	Suggested function(s)	Reason
Maintain a constant climb angle	<code>gamma_hold</code>	Simple PD on flight-path angle
Intercept a maneuvering target	<code>augmented_pro_nav</code> or <code>sliding_mode_guidance</code>	Handles LOS-rate changes robustly
Fuel-optimal rendezvous	<code>cw_lqr</code> (with tuned Q/R) or <code>mpc_rendezvous</code>	Explicit quadratic-cost formulation
Fast time-optimal capture	<code>lead_angle</code> or <code>zero_effort_miss</code>	Provides aggressive lead to shorten miss distance
General nonlinear discovery	Include <i>all</i> functions in the library	Gives SINDy a rich candidate set to pick the most explanatory terms

Feel free to edit any of the gains, constants, or add more parameters (e.g., target orbital rate n for a true CW model). The functions are deliberately written **stateless** and vectorised, so they work efficiently with large training datasets.

If you need a **different state ordering** (e.g., `[vx, vy, vz, x, y, z]`) just adjust the indexing in each routine or write thin wrappers that reorder the array before calling the core function.